

Consolidation Projects — EDA, Feature Engineering, Classification & Regression

A brush-up set for Géron Chapters 2–4

Data links + reference benchmarks included

How to use this set

Five small projects that, between them, re-exercise everything from the last few days: EDA, the feature-engineering pipeline, classification metrics, regression, regularization, and bias/variance. They deliberately **reuse data across projects** so you don't sprawl, and they stay in your domain so the practice compounds toward your real positioning.

Keep your standing rule in view: scope each one small, finish it, write a one-paragraph notes entry, move on. If you only have bandwidth for two, do **P2 and P3** — together they cover classification, regression, regularization, and your domain evaluation rules in a single pass. Add **P5** third; it pays forward into PyTorch.

Each project lists **where to get the data** and a **reference benchmark** so you can judge whether your model is actually working — not just whether it runs.

P1 — Clinical-outcome prediction (*the EDA + feature-engineering muscle*)

Drills: EDA, missing-value imputation, encoding categoricals, scaling, ColumnTransformer + Pipeline, cross-validation. This is a *mixed-type tabular* problem on purpose — scRNA matrices are all-numeric and won't exercise the encoding/imputation plumbing.

Multiclass? Optional. The standard target is **binary** (disease present / absent). The raw label is graded **0–4 by severity**, so predicting severity instead gives you a 5-class *ordinal* multiclass problem (the classes are ordered, which plain softmax ignores — a useful subtlety to notice).

Dataset — UCI Heart Disease (Cleveland), 303 patients, 13 features:

```
# pip install ucimlrepo
from ucimlrepo import fetch_ucirepo
heart = fetch_ucirepo(id=45)          # dataset id 45
X = heart.data.features               # mixed numeric + categorical
y = heart.data.targets               # 0 = absence, 1–4 = severity
# binary version: (y > 0).astype(int)
```

- Source: <https://archive.ics.uci.edu/dataset/45/heart+disease>
- Cleaned single-CSV mirror (no missing values, pre-binarized): <https://www.kaggle.com/datasets/cherngs/heart-disease-cleveland-uci>

Reference benchmark (binary): with a proper pipeline and cross-validation, logistic regression on Cleveland commonly lands around **80-88% accuracy** (some careful studies report up to ~90%), with ROC-AUC roughly **0.85-0.90**. SVM and tree ensembles sit in a similar band.

Red flag for this dataset: papers reporting **95-99%** accuracy almost always leak — they tune on the test set, oversample *before* splitting, or use the tiny 303-row set without honest CV. If you see 0.97, suspect leakage before celebrating. Hitting a *believable* 0.84 with a clean pipeline is the real win here.

P2 — Cell-type annotation (*the classification-metrics project*)

Drills: softmax/logistic classification, and every classification-metric lesson from your notes — donor/batch-aware splitting, macro-F1, PR curves, confusion-matrix error analysis through a biological lens.

Multiclass? Yes — this is the multiclass centerpiece. ~10 cell types; the confusion matrix becomes a K×K biological map.

Dataset — pbmc68k_reduced (700 cells × 765 genes, cell-type labels built in):

```
import scanpy as sc
adata = sc.datasets.pbmc68k_reduced()
X = adata.X                                # already HVG-reduced, log-normalized
y = adata.obs["bulk_labels"]               # ~10 cell types
# bigger option with labels: sc.datasets.pbmc3k_processed() (louvain-derived types)
```

- Docs: https://scanpy.readthedocs.io/en/stable/generated/scanpy.datasets.pbmc68k_reduced.html
- For a donor/batch-split exercise, grab a multi-donor set from scvi-tools tutorials: <https://docs.scvi-tools.org/en/stable/tutorials/index.html>

Reference benchmark: the production tool for exactly this task — **CellTypist** — is itself **logistic regression trained with SGD**, so you're rebuilding the real thing. On hard *cross-tissue atlas* data, a tuned CellTypist model reaches a **macro-F1 around 0.73**; deep models (scTab) and tuned XGBoost reach **~0.81**. On a clean, broad-type PBMC set like this one, expect **accuracy above 0.90** — but **watch macro-F1 fall below accuracy** because the small/rare types drag it down. That gap is the lesson: report macro-F1, not accuracy.

You're on track if: accuracy > 0.85, macro-F1 visibly lower than accuracy, and the confusion matrix's worst cells are *biologically adjacent* types (e.g. CD4 vs CD8 T cells), not random.

P3 — Phenotype-from-expression regression (*the Chapter 4 + regularization project*)

Drills: the $p \gg n$ regression regime where regularization is mandatory — Ridge vs Lasso vs ElasticNet, the alpha sweep, and watching which genes Lasso zeros out.

Multiclass? No — this is **regression**, the deliberate counterweight so you’re not doing classification five times. (You *can* bin the target into groups to make it multiclass, at the cost of discarding information.)

Dataset: reuse P2’s expression matrix; predict a **continuous** target. Cleanest self-contained option: compute a **signature/module score** from a held-out gene set (e.g. a cell-cycle or interferon-response panel) and predict it from the *rest* of the transcriptome. For a genuine “transcriptomic age” clock you need donor metadata — pull a multi-donor set from scvi-tools or CELLxGENE (<https://cellxgene.cziscience.com/>).

```
import numpy as np
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.dummy import DummyRegressor
# y_cont = a continuous module score; X = remaining genes (standardized)
```

Reference benchmark — the baseline IS the benchmark here. With thousands of gene-features and few samples, plain LinearRegression overfits catastrophically (near-perfect train R^2 , garbage val R^2) — that failure is the point. The honest check is whether your regularized model **beats a DummyRegressor that just predicts the mean**:

You’re on track if: (1) unregularized linear regression shows a huge train-vs-val R^2 gap; (2) Ridge/ElasticNet beat DummyRegressor (val R^2 clearly > 0); (3) sweeping alpha from low→high makes the train-val gap *shrink then the val score peak then fall*; (4) Lasso zeros out most genes and keeps a biologically plausible handful.

P4 — The rigor harness (*bias/variance + evaluation discipline*)

Drills: leakage-safe evaluation as a reusable artifact — learning curves, nested CV, regularization sweeps with the train-val gap plotted.

Multiclass? Inherits whatever you point it at — feed it P2 and it evaluates a multiclass problem (just make sure it computes **macro-averaged** metrics).

Dataset: none new — it wraps P2 and P3. The deliverable is a small, well-structured, tested module you’ll actually reuse (and which doubles as a public, shipped artifact for your career goal).

Reference benchmark: there’s no dataset score here; the “benchmark” is whether the harness **reproduces a known pathology on demand** — can you make it *show* overfitting (train $\gg$ val) and underfitting (both low, close) by turning model complexity up and down? If your learning curves visibly cross the way Géron’s figures do, the harness works.

P5 — Softmax-from-scratch (*the math-deepening bridge to PyTorch*)

Drills: softmax + cross-entropy + batch GD + early stopping, by hand in NumPy. Géron’s own Chapter 4 exercise. This skeleton is a PyTorch training loop minus autograd — the single highest-leverage item here for where you’re headed.

Multiclass? Yes, by construction — run it on **3+ cell types**, or you’ve just rebuilt binary logistic regression.

Dataset: a 3–4 class subset of P2’s pbmc68k_reduced.

Reference benchmark: your hand-rolled model, validated against `sklearn.linear_model.LogisticRegression` on the *same* subset.

You’re on track if: your from-scratch accuracy and loss land **within ~1-2% of sklearn’s** on the same split, and your training-loss curve decreases monotonically (if it oscillates or diverges, your learning rate is too high — the classic GD-needs-scaling lesson from Chapter 4).

How to know *any* model is “good enough”

Four checks, in order — this is the reusable mental harness:

1. **Beat the dumb baseline.** `DummyClassifier(strategy="most_frequent")` / `DummyRegressor(strategy="mean")` is the floor. If you don’t clearly beat it, nothing else matters. On imbalanced data the dummy’s *accuracy* can look high — which is exactly why you use macro-F1.
2. **Use the right metric.** Regression → RMSE/MAE/R². Classification → **macro-F1** (not accuracy) on imbalanced data; PR curves for rare classes.
3. **Compare to published numbers** (the per-project benchmarks above) — but only as a sanity band, since splits and preprocessing differ.
4. **Distrust scores that are too good.** On small or leaky datasets, near-perfect scores signal a methodology bug, not a great model. A believable, honestly-validated number beats an inflated one every time.

Coverage map

	EDA	Feat. eng.	Classification	Regression	Regularization	Bias/var	Multiclass
P1 Clinical	●●	●●	●	○	○	●	optional
P2 Cell type	●	○	●●	—	○	●	yes
P3 Pheno- type	●	○	—	●●	●●	●	no

	EDA	Feat. eng.	Classification	Regression	Regularization	Bias/var	Multiclass
P4 Rigor har- ness	○	—	●	●	●	●●	inherited
P5 Soft- max scratch	—	—	●●	—	—	○	yes

●● *core* · ● *touched* · ○ *light* · — *n/a*

All data links in one place

- **UCI Heart Disease (Cleveland):** <https://archive.ics.uci.edu/dataset/45/heart+disease> · cleaned CSV: <https://www.kaggle.com/datasets/cherngs/heart-disease-cleveland-uci>
- **scanpy pbmc68k_reduced:** https://scanpy.readthedocs.io/en/stable/generated/scanpy.datasets.pbmc68k_reduced.html
- **scanpy pbmc3k / pbmc3k_processed:** <https://scanpy.readthedocs.io/en/stable/generated/scanpy.datasets.pbmc3k.html>
- **scvi-tools tutorial datasets:** <https://docs.scvi-tools.org/en/stable/tutorials/index.html>
- **CellTypist (the LR-based production analogue of P2):** <https://www.celltypist.org/>
- **CELLxGENE (multi-donor data with metadata):** <https://cellxgene.cziscience.com/>